



Predicting the Next Response: Demonstrating the Utility of Integrating Artificial Intelligence-Based Reinforcement Learning with Behavior Science

David J. Cox^{1,2}  · Carlos Santos¹

Accepted: 21 February 2025

© Association for Behavior Analysis International 2025, corrected publication 2025

Abstract

The concepts of reinforcement and punishment arose in two disparate scientific domains of psychology and artificial intelligence (AI). Behavior scientists study how biological organisms *do* behave as a function of their environment, whereas AI focuses on how artificial agents *should* behave to maximize reward or minimize punishment. This article describes the broad characteristics of AI-based reinforcement learning (RL), how those differ from operant research, and how combining insights from each might advance research in both domains. To demonstrate this mutual utility, 12 artificial organisms (AOs) were built for six participants to predict the next response they emitted. Each AO used one of six combinations of feature sets informed by operant research, with or without punishing incorrect predictions. A 13th predictive approach, termed “human choice modeled by Q-learning,” uses the mechanism of Q-learning to update context-response-outcome values following each response and to choose the next response. This approach achieved the highest average predictive accuracy of 95% (range 90%–99%). The next highest accuracy, averaging 89% (range: 85%–93%), required molecular and molar information and punishment contingencies. Predictions based only on molar or molecular information and with punishment contingencies averaged 71%–72% accuracy. Without punishment, prediction accuracy dropped to 47%–54%, regardless of the feature set. This work highlights how AI-based RL techniques, combined with operant and respondent domain knowledge, can enhance behavior scientists’ ability to predict the behavior of organisms. These techniques also allow researchers to address theoretical questions about important topics such as multiscale models of behavior and the role of punishment in learning.

Keywords Reinforcement learning · Artificial intelligence · Simulations · Theory testing

The importance of consequences to change behavior has been known at least since 2400 BCE via the Code of Urukagina (Finegan, 2019). However, scientific approaches to understanding the mechanisms around *how* behavior changes as a function of consequences arose only recently (e.g., Thorndike, 1898). For behavior scientists, these were most notably outlined by Skinner and colleagues via reinforcement, punishment, and extinction in the 1930s and 1940s (Ferster & Skinner, 1957; Skinner, 1938). Today, this framework allows researchers and practitioners to predict the change in behavior of biological organisms spanning invertebrates (e.g., Kandel, 1979; Hawkins et al., 2006) to humans (e.g., Ayllon & Michael, 1959). If the criterion is only that the environment shapes behavior, organisms evolutionarily further from humans might also be included, such as bacteria (e.g., Adler, 1966), plants (e.g., Baluška & Mancuso, 2009; Hedrich & Neher, 2018), and even Archaea (Albers & Meyer, 2011; Valentine, 2007).

In parallel, computer scientists have also pursued the concepts of reinforcement and punishment in the domain of artificial intelligence (AI; Farley & Clark, 1954; Minsky, 1954). Potentially inspired by the obvious role of consequences to adapt behavior in biological systems, early researchers developed algorithms so that non-biological, artificial systems could adapt their behavior through environmental feedback (e.g., Michie, 1961; Samuel, 1959). This was more formally outlined later via mechanisms of reward and punishment in reinforcement learning (RL; Sutton & Barto, 2018). AI models also demonstrate predictable changes in behavior based on interactions with the environment, leading to often human-like or better performance in tasks such as game playing (e.g., Silver et al., 2016; Tesauro, 1995) and robotic control (e.g., Kober et al., 2013). In short, artificial systems can be built to exhibit adaptive behaviors influenced by environmental consequences.

Despite the many conceptual similarities between adaptive behavior in biological and artificial systems, many important differences exist between the two fields in the approach, assumptions, goals, and end purpose. What both domains share, however, is the fundamental finding that robust, adaptive systems can be designed by providing feedback to the behaving system about its behavior relative to some goal. Behavior scientists are often interested in how biological systems change their behavior based on feedback (experimental analysis of behavior), how this knowledge can be used to improve the lives of living organisms (applied behavior analysis), and how the correspondence truths identified through research and practice allow us to extend what we know into unexplored domains (behaviorism). To help behavior scientists add tools to their toolbox for understanding the adaptive behavior of behaving systems, this article provides a brief primer on the unique scientific history of RL in AI, its commonalities and differences to operant learning in behavior science, and how combining insights from each can advance research in each domain.

Reinforcement Learning in Artificial Intelligence

Many modern RL approaches include solutions to the early challenges researchers faced when building artificial systems that learn. We will highlight an example below that demonstrates each concept. However, we will first describe each component and its utility when trying to build artificial systems that learn.

Some Fundamental Components

One fundamental component is termed *stochastic learning*. *Stochastic learning* is a process whereby the behavior of a system (e.g., a biological organism, an artificial organism) develops over time (e.g., behavior today might differ from yesterday) and according to probabilistic principles (e.g., changing response strength via contact with intermittent schedules of reinforcement; Ifenthaler & Seel, 2012). Stochastic learning incorporates randomness and probability into the learning process, allowing systems to make decisions based on uncertain information (Minsky, 1954). This approach is crucial in RL because it enables AI systems to respond and adapt appropriately to the unpredictability of nonlaboratory environments. Using probabilistic methods, AI systems can better adapt and optimize their strategies over time, leading to more robust and intelligent¹ behavior.

A second fundamental concept is self-organizing systems. Self-organizing systems are designed to autonomously arrange their internal structures and behaviors in response to external circumstances without centralized control to stabilize the structure or function of the system against external fluctuations (Banzhaf, 2009; Farley & Clark, 1954). Creating self-organizing systems is pivotal in RL because the goal is to build a system that does not require an outside party to manually update the system in response to the environmental conditions it encounters. That is, making AI-systems self-organizing enables them to adapt and evolve in complex, dynamic environments where the programmer is likely unaware of the totality of contexts and environmental characteristics the artificial system will contact.

A third fundamental concept is the credit assignment problem. *Credit assignment* can be defined as the process whereby a system attributes the delivery of a reinforcer or punisher to specific actions the system emitted (e.g., Sammut, 2011). In other words, behaving systems often emit many different behaviors leading up and temporally coinciding with the delivery of a reinforcer or punisher. *Credit assignment* involves the behaving system learning which actions it emitted that led to the reinforcer or punisher and which actions did not. It is possible to learn inaccurate relations between behavior and consequences (e.g., superstitious behavior). Therefore, when building an AI system, this is often framed as the *credit assignment problem* because the system has to learn how to accurately associate reinforcers and punishers only with the actions they truly are contingent upon (Minsky, 1961). Of note, the credit assignment problem has been discussed in behavior science and remains a challenge to explain for biological organisms (Hutchison, 2012; Staddon, 2001; Staddon & Zhang, 2017; Touretzky & Saksida, 1997).

A Simple Artificial System that Learns

One example of how this all coalesced into a single system early on is the Machine Educable Noughts and Crosses Engine (MENACE; Sutton & Barto, 2018). MENACE,

¹ Intelligence can, and has been, defined in many different ways. Here we are simply referring to the Latin root base of intelligence, *intellegerere* meaning *inter* (between) + *legere* (choose). Restated, intelligence here is defined as “the ability to choose.”

also known as the tic-tac-toe engine (TTTE) was built from matchboxes with every possible game state drawn on a unique matchbox (Michie, 1961). Inside each matchbox were differently colored beads corresponding to a move the MENACE system could make in that game state. To make a move during a game, the matchbox associated with the game state was found, the matchbox shaken, and a single bead drawn to determine the move played. This process was repeated until the game was over.

At the end of the game, feedback was then provided to the matchbox system. If MENACE lost, the beads chosen throughout the game were removed, making it less likely that those responses would be emitted in the future in the corresponding game state. If MENACE tied, each bead was put back in its respective box such that there was no change in the probability of emitting that response in the future in that game state. If MENACE won, three beads of the emitted response were added to the corresponding box, making it more likely that response was emitted in the future in that game state. This simple algorithmic method for changing the response probabilities within each game state was sufficient to “teach” the system to play—and win—the game of tic-tac-toe.

It may help to make explicit how the fundamental components mentioned above were incorporated into the design of MENACE. Stochastic learning was incorporated in two ways. First, the response emitted by the system was randomly drawn from a distribution of beads corresponding to different response alternatives. Second, the human player MENACE was playing against could emit many different next moves following the move MENACE made. That is, there is uncertainty over how the system will change after a response MENACE made. MENACE can be termed self-organizing because the system can independently change the likelihood that different responses are emitted in each state over time. MENACE was programmed with a simple rule for how to do that by removing (if it lost), putting back (if it tied), or increasing the number of beads by three (if it won). But, the simple rule then allows the system to update itself automatically. Last, the credit assignment problem was solved by equally distributing reinforcement or punishment across all responses within the entire chain of responses that led to a game outcome (i.e., response distributions at the end of the game changed the same as those at the beginning of the game).

Since this early work, many variants of RL-based artificial systems have been developed involving one or more of these components (Sutton & Barto, 2018). At its core, however, all RL has similar selective mechanisms to adapt a system’s behavior using reward and punishment. Though RL in AI carries conceptual similarities to the selectionist accounts of reinforcement and punishment in biological organisms, there are many important differences. In the next section, we provide a general and high-level outline and demonstration of how this all works to make the similarities and differences easier to see.

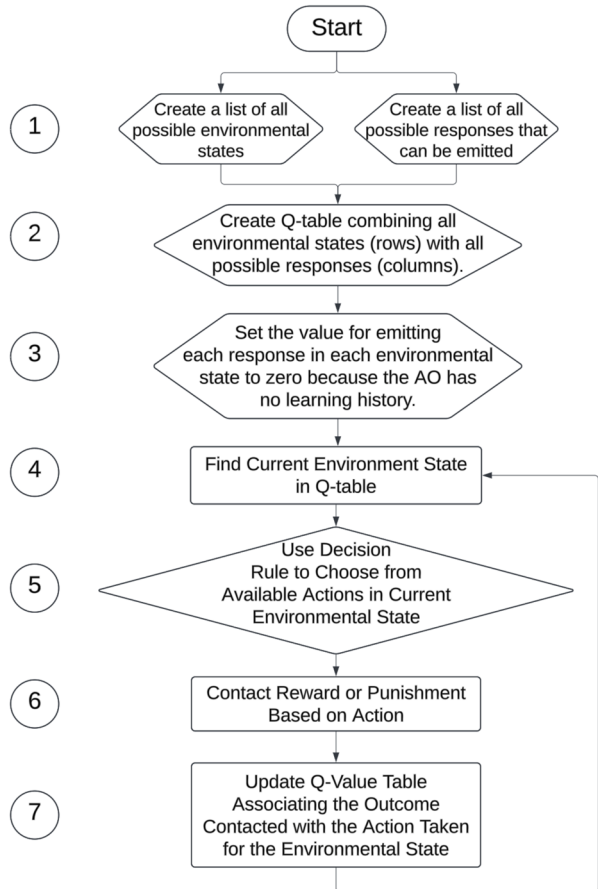
An Example Modeling a Matching Law Experiment

To create an artificial system that learns, the system has to be programmed with rules for how it will interact with the environment over time. Figure 1 shows a flowchart of the basic idea for one type of RL algorithm termed Q-learning which

is a form of model-free reinforcement learning (Watkins & Dayan, 1992). As highlighted in greater detail below, Q-learning was chosen because the algorithmic process instantiated by Q-learning can animate an artificial organism (AO) in a manner that is likely to be intuitive to behavior scientists familiar with the choice literature and the notion of quantifying reinforcer value within the functionally defined four-term contingency.

To set up a Q-learning experiment, the researcher must first do several preparatory things denoted by the ○ shape in Fig. 1. First, they need to identify the possible set of environmental states that the AO can contact within an experiment and the total set of possible responses the AO can emit across all of those environmental states (Graesser & Keng, 2019). For example, if describing a matching law experiment, the total set of environmental states could be the list of VI-schedule values concurrently present throughout the experiment, such as VI 60 s–VI 10s, VI 40 s–VI 20 s, VI 30 s–VI 30 s, VI 20 s–VI 40 s, and VI 10 s–VI 60 s (five in total). The list of all possible responses the researcher cares about could be right lever presses and left lever presses (two in total).

Fig. 1 Flow Chart Corresponding to a Generic Flow One Type of RL Algorithm Called Q-Learning



The second step is to combine these two lists into a table where the set of environmental states comprises the rows, and the available responses are the columns (Graesser & Keng, 2019). The top of Fig. 2 shows what this would look like using the matching law example described in the preceding paragraph. As a final preparatory step, in each cell within the table, the initial value of emitting each response in each environmental context is set to 0 because the AO does not have any learning history with emitting either response in the environmental contexts arranged.

Once the preparatory steps are completed, the AO can begin learning by emitting responses, gaining feedback, and updating the value of emitting a specific response in a specific context. This process starts by first identifying what environmental state the AO is in (step 4, Fig. 1) and finding the corresponding row in the Q-table (Fig. 2). For the simplified matching example in use here, this would correspond to the concurrent VI arrangement associated with the current condition. Let's say it is the top row of the Q-table in Fig. 2 where the left key is programmed to lead to food on a VI 60-s schedule, and the right is programmed to lead to food on a VI 10-s schedule.

The next step is for the AO to emit one of the available response options based on the current context (Step 5, Fig. 1). There are many different ways this decision can be programmed to occur, and we get into more details about this below. But, in many instances, this decision involves balancing two strategies (e.g., Graesser & Keng, 2019). First, and coinciding with the fundamental assumptions of behavioral economics (e.g., Hursh & Silberberg, 2008), the AO could choose based on the experienced expected value of all options. This is also known as *exploiting* the knowledge of the state-action-reward pairings learned. This could be accomplished by always choosing the response associated with the higher value, by weighting the probability of each option based on their relative values, or whatever arrangement the programmer might want to experiment with. The second strategy is termed *exploration*, which involves the AO randomly choosing from the available response options.² The probability with which the AO explores versus exploits is a parameter that can be fixed or allowed to dynamically or strategically change during a simulation.

The next steps are for the AO to contact reinforcement or punishment based on the response emitted (Step 6, Fig. 1) and to update the corresponding value in the Q-table (Step 7, Fig. 1). For example, consider a first response to the right lever after 10 s have passed within the experimental condition listed in the first row of tables in Fig. 2. If the response led to 5-s access to grain, we could update the value of a "Right Lever" by +5 under the environmental condition "VI 60 s; VI 10 s."

In this matching example, the surrounding environment has not changed. And, so the following cycle would continuously repeat for the duration of an experimental session: (1) find the row in the Q-table corresponding to the current environment; (2) use the decision algorithm to emit a response; (3) contact reward, punishment, or

² NB: The probability distributions here can be weighted however the programmer wants. For example, all available response options could have the same probability of being chosen, or each response option could be weighted such that responses that have not been emitted in a while have a higher probability of being explored compared to responses emitted more recently.

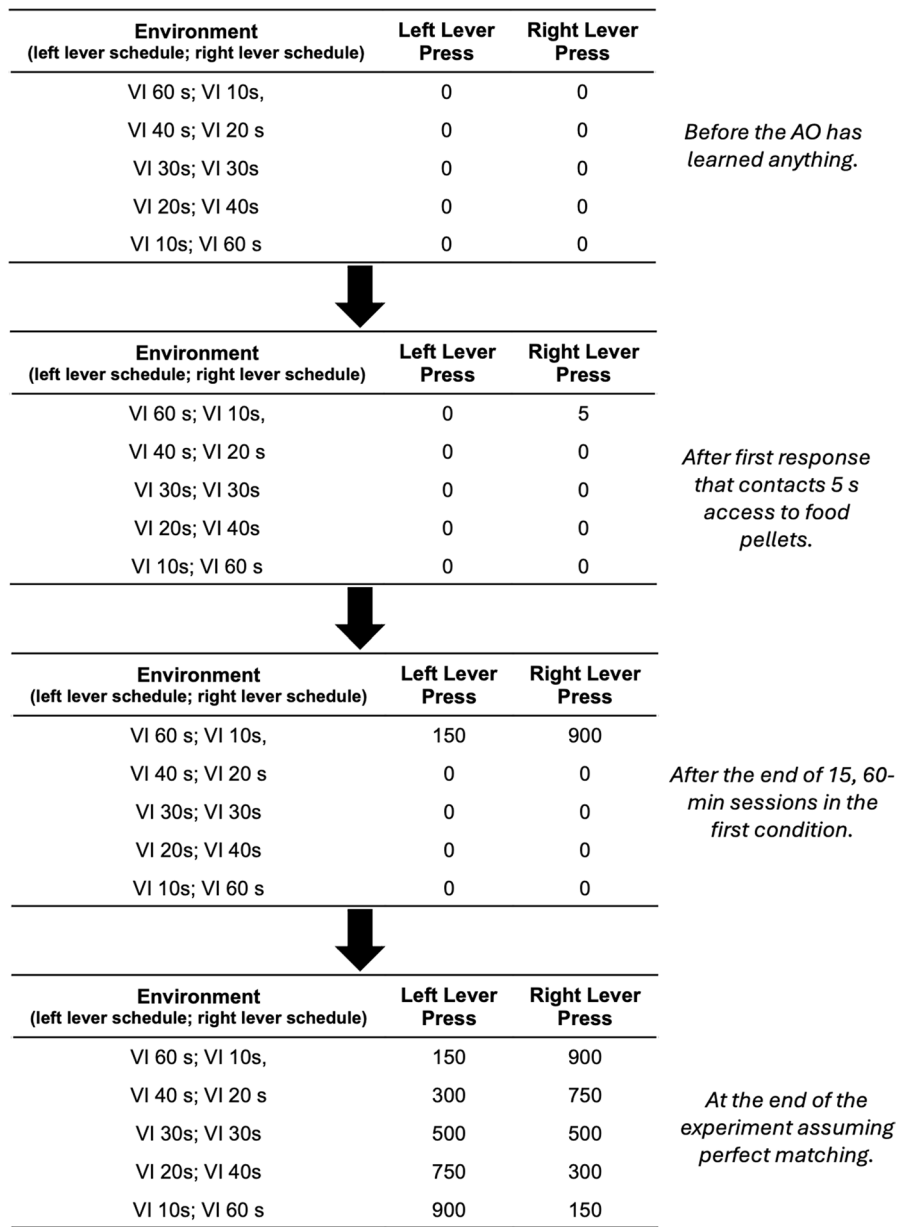


Fig. 2 An Example Q-Learning Table for an Experiment Involving Five Combinations of Concurrent VI Schedules. *Note:* The top table shows the Q-table at the onset of an experiment or simulation. The second table shows how the table would update after the first response that contacts reinforcement of a value equal to 5. The third panel shows what the table would look like after completing the condition described in the first row. The bottom panel shows the values at the end of a matching experiment

nothing; (4) update the correct Q-table value when a reinforcer or punisher is contacted. Over time, this process results in the Q-table containing the cumulative experienced value associated with emitting each response within a context (e.g., third table, Fig. 2). When repeated across many contexts, the Q-table eventually contains the associated value of emitting all available responses in all available environmental contexts (bottom table, Fig. 2).

A Few More Technical Notes

The above was described simply and via analogy to a matching law experiment that readers are likely to be familiar with. There are a few additional components RL programmers often include to get AOs to model and solve real-world problems efficiently. First, programmers need to give the AO an objective. This typically is to maximize the reward obtained over time ($\max_{\pi} E[R_0 + R_1 + \dots + R_t | \pi]$; Graesser & Keng, 2019). However, in a technical sense this could be any objective (e.g., minimize points lost; maximize in one context, sample repeatedly in another) and is defined by the programmer based on the real-world situation they are trying to model.

Second, the programmer needs to choose what the AO will learn. For example, should the AO learn how to choose actions based on the current state (i.e., a discrete probability distribution over an action space— $\pi_{\theta}[a|s]$)? Should the AO learn the expected reward value associated with each action given an environmental state (V^{π} or $Q^{\pi}[s, a]$)? Should the AO learn how the environment transitions from state to state ($[r, s'|s, a]$)? Or, a combination of these three? Answering these three questions will determine which type of learning algorithms you would implement in your AO (for book-length treatments, see Graesser & Keng, 2019 or Sutton & Barto, 2018).

Last, the matching example above was fairly simple compared to many RL use cases. In particular, the matching law example involved five unique environmental states, two responses, and the organism did not change between environmental states except at condition changes. Many nonlaboratory environments where RL algorithms are used can involve dozens to thousands of environmental states, hundreds or thousands of response options, and emitting a response in one environmental state might move the organism to a different environmental state altogether (see below example).

Finally, to finish off the description of the Q-learning algorithm, we need to add a slight bit of technical complexity (Fig. 3). Above, we discussed how we add the reward contacted following each response to the existing value of emitting that action in that context (“Current Q-Value” + “Reward in Current State”; Fig. 3). However, because being in some states is more valuable than being in others, we also want to capture how chains of actions affect the amount of future rewards likely to be contacted. To do this, we add the difference between the “highest Q-values following possible actions from the new state” and the “current Q-value” from the current state the AO is in (Fig. 3). In addition, those future rewards are discounted (“Discount Rate”; Figure 3) in a manner conceptually similar to discounting in

behavior analysis though modeled differently (Rachlin, 2006; Graesser & Keng, 2019). Finally, how fast the AO learns in the direction of the feedback is controlled by the learning rate (α ; Fig. 3). This entire process can be described algorithmically and expressed as the Bellman equation (Fig. 3) (Graesser & Keng, 2019).

Q-learning is only one type of RL algorithm that can be used. It is a value-based algorithm because it tries to learn the associated value of each action in each state and was chosen to highlight here because it operates seemingly most like the fundamental assumptions of behavioral economics (Hursh & Silberberg, 2008). Other ways of implementing this basic sequence exist with most sharing the same similarities and differences as how we might approach learning in biological organisms (Graesser & Keng, 2019; Sutton & Barto, 2018).

Similarities and Differences with RL in Behavior Science

For a full-length chapter specific to the similarities and differences between RL in AI and behavior science, readers are referred to Cox et al. (2023). In brief, Fig. 4 shows a schematic of the three-term contingency commonly used in behavior science to describe the processes and mechanisms whereby a biological organism's behavior might be changed. Further, Table 1 compares and contrasts common terms from behavior science and RL.

As behavior scientists working with biological organisms, we often can only manipulate events antecedent and consequent to behavior (solid lines, Fig. 4). The total set of possible actions and environmental states are unlikely to be fully known. We often measure some dimensions of some of all possible actions (e.g., rate, latency, duration), and we measure some dimensions of some of all possible environmental stimuli (e.g., hue, intensity, duration of key lights, or vocal discriminative stimuli). Reinforcer and punisher values are known imprecisely and only relative to whatever is concurrently available and in trajectory to that point in time (e.g., Baum, 1974; Davison & Cowie, 2022). We are often interested in then using the above information to describe and predict how biological organisms do behave as they try to meet their many biological needs. The complexity of the total biological system,

$$Q_{new}(s, a) = \underbrace{Q(s, a)}_{\text{Current Q-Value}} + \underbrace{\alpha}_{\text{Learning Rate}} \cdot \left[\underbrace{R(s, a)}_{\text{Reward in Current State}} + \underbrace{\gamma}_{\text{Discount Rate}} \cdot \underbrace{\max Q'(s', a')}_{\text{Highest Q-values following possible actions from new state.}} - \underbrace{Q(s, a)}_{\text{Current Q-Value}} \right]$$

Fig. 3 Q-Learning Bellman Equation

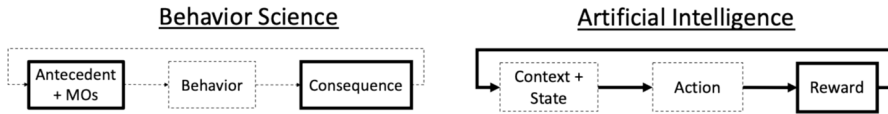


Fig. 4 Similarities and Differences in Reinforcement Learning Using the Three-Term Contingency from Behavior Science (left panel) and Its Counterpart Jargon in RL within AI (right panel). *Note.* The thick lines (boxes and arrows) highlight the primary areas manipulated to change behavior toward a desired end by researchers and practitioners in that domain

however, makes it unlikely we are accounting for everything functionally related to the total repertoire of the organism.

In contrast, AI researchers building RL systems typically manipulate different aspects of the three-term contingency than behavior scientists (solid lines, Fig. 4). Working with artificial organisms, we often manipulate or learn exactly how a context leads to an action being emitted. We know exactly how an action leads to a reward. We know the exact reinforcer or punisher value. And we know exactly how a reward updates the internal state of the artificial organism. This is often because the overall scope of what the artificial organism needs to accomplish is well defined, contains fewer dimensions, and carries the goal of maximizing reward for a singular use case as opposed to keeping a complex biological organism alive. In short, AI scientists and behavior scientists share similar conceptual components when studying RL in their chosen systems. However, they manipulate different aspects of organism-behavior-environment system and have different goals.

Mutual Utility?

Given that these two disparate scientific domains have different goals and methods, you might wonder how each domain could benefit from bridging these two fields. In the remainder of the paper, we will walk through one example using the same Q-learning algorithm described above to build AOs that can predict the next response emitted by humans in a complex, dynamic, nonlaboratory context. To do this, the first author chose the socially significant context of Major League Baseball (MLB) as there were five reasons it might be useful to predict the next response emitted by a professional baseball player.

The first two benefits surround our ability to map the RL training process to the game of baseball. First, building accurate RL models often requires a lot of data. Statcast was introduced to all MLB stadiums in 2015 as an automated tool developed to collect data and analyze player movements (MLB, 2024). Since then, baseballsavant.mlb.com has maintained a large and open-source database of every pitch thrown, the behavior of all players during each play, and the resulting outcomes of each play. This makes available the large datasets needed to build accurate RL models. Second, unlike other sports with more continuous gameplay (e.g., basketball, hockey), each pitch thrown is a discrete response with a natural temporal break from

Table 1 Common Analogous Terms from Behavior Science and RL with Examples

Concept	Operant and Respondent Conditioning	Reinforcement Learning (AI)
Organism / Agent	<i>Biological Organism</i> (Examples: A rat in an operant chamber, a child learning to read, a dog in agility training)	<i>Agent / Artificial Organism</i> (Examples: A robot learning to walk, a self-driving car adjusting speed, a chess-playing AI system)
Environment	<i>Physical or social surroundings</i> (Examples: A classroom, a zoo, a home)	<i>Simulated or real-world environment</i> (Examples: A robotic lab, an online recommendation system, a video game simulation)
Behavior/Action	<i>Response</i> (Examples: Pressing a lever for food, answering a math problem, pulling a door handle)	<i>Action</i> (Examples: A robotic arm picking up an object, an AI system selecting a move in chess, a drone changing altitude)
Stimulus/State	<i>Antecedent Stimulus</i> (Examples: A bell before food delivery, a red light when driving, a verbal command in dog training)	<i>State</i> (Examples: The position of pieces on a game board, a sensor reading, a customer's browsing history)
Reward/ Reinforcement	<i>Reinforcement</i> (Examples: Giving a high-five for sitting, getting a bonus at work, seatbelt ding turning off when you buckle)	<i>Reward Function</i> (Examples: Gaining points in a game, reducing energy consumption in a smart thermostat, winning a chess match)
Aversive/ Punishment	<i>Punishment</i> (Examples: A timeout for misbehavior, getting burned from touching a hot stove, losing privileges for being late)	<i>Negative Reward</i> (Examples: Losing points in a game, getting a penalty in an AI-based trading system, increasing energy consumption in a smart grid)
Extinction	<i>Extinction</i> (Example: A child stops raising their hand when ignored, a dog stops begging if they are not fed from table, a pigeon stops pecking if food is not delivered)	<i>Lack of Reward Leading to Decrease in Action Selection</i> (Examples: A robot stops trying a failed strategy, a recommendation system stops suggesting low-rated items, a chess AI stops using an ineffective move)
Stimulus Generalization	<i>Learned Response Emitted in New Context(s)</i> (Examples: A child applies math skills in real life, a dog responds to different variations of "sit", a rat presses levers under varying hues of green)	<i>Generalizing from Observed States</i> (Examples: A chess-playing AI responds to unseen positions, a self-driving car responds to unfamiliar road layouts, a robotic arm manipulates new objects)
Learning Rule	<i>Contingency-Based Probability Increase</i> (Examples: Higher rate of lever presses when followed by pellets, a child reads more when praised, a dog fetches when given treats)	<i>Update of Action Values Based on Reward Feedback</i> (Examples: AI refines robot path planning, a chess-playing AI learns optimal strategies, a chatbot adjusting dialogue responses)

the preceding play. This is beneficial because many RL algorithms operate in discrete timesteps, as described above, making their application to the baseball context straightforward.

The third and fourth reasons are specific to the goals of research in behavior analysis. Behavior analysts are often interested in understanding the variables that allow us to describe, predict, and understand the behavior of individual organisms. The Statcast database contains data on the antecedent, response, and game-related consequences at the individual player level, allowing behavior analysts to generate RL models of individual human behavior. Further, the field of behavior analysis has a history of describing choice in baseball contexts (Cox et al., 2017; Cox et al., 2021; Falligant et al., 2021). Thus, we understand the likely reinforcers and contextual variables influencing choice during baseball games. This allows us to constrain what information the artificial organisms use to only those relevant from an operant perspective.

Fifth, though past research in behavior analysis suggests that the generalized matching equation (GME) describes choice well in baseball contexts, it is unclear what practical utility that has. In particular, the GME describes the relative allocation of pitches across 70–80+ pitches thrown during a game (Cox et al., 2017; Cox et al., 2021; Falligant et al., 2021). Batters see an average of three to four pitches per at-bat (MLB, 2024). The utility of current operant-based modeling applied to baseball would be greater if the next pitch thrown could be predicted rather than knowing general trends spanning the entire game. RL algorithms provide an opportunity to move to molecular, next-pitch predictions in a way that no other operant models in this sports context have yet accomplished.

Predicting the Next Pitch

Basic Preparation

As before, we first defined the set of all possible finite states and actions. The set of state variables that could be defined before each pitch was thrown consisted of: (1) whether the batter stood right-handed or left-handed; (2) the number of balls in the count (0–3); (3) the number of strikes in the count (0–2); (4) the number of outs (0–2); (5) the presence of runners on first base, second base, and third base; and (6) the difference in score between the pitching team and opposing team (winning, losing, tied). Readers unfamiliar with baseball and interested in learning more are referred to the Public Broadcasting System (2024). Readers unfamiliar with baseball and who could care less about these details, no worries. The main point here is that we can identify a set of stimulus conditions that pitchers contact during a baseball game, have some probability of influencing choice, and can be explicitly labeled and quantified. All combinations of these six features lead to 1,728 unique game states ($2 \times 4 \times 3 \times 3 \times 2 \times 2 \times 2 \times 3 = 1,728$).

To define the total set of actions, we can use work from past researchers who provide a functional set of possible actions (Cox et al., 2017; Cox et al., 2021; Falligant et al., 2021). Response options are pitches that are hard and straight (e.g.,

fastball), hard and breaking (e.g., slider), slow and straight (e.g., change-up), and slow and breaking (e.g., curveball). On each pitch, the goal is to predict which of these four categories the pitcher is likely to throw as it creates our action space. Just like in Fig. 2, we can create a 4 x 1728 Q-table to keep track of the expected value of throwing each pitch within each of those game contexts.

The reward function in RL can be conceptualized similarly to schedules of reinforcement in behavior science (Ferster & Skinner, 1957; Morgan, 2010). For the AOs in this experiment, we programmed a reward of +1 for each instance in which it correctly predicted the pitcher's next response. We then set up two possible outcomes for each instance in which the AO incorrectly predicted the pitcher's next response. As with much research in behavior science, we programmed for no change in stimulus conditions following each incorrect prediction for one AO. The second AO contacted a penalty of -1 following each incorrect prediction (functionally serving as punishment). As is typical of RL work, the objective for the AO was to maximize overall rewards. In this case, maximizing the rewards obtained occurred through the AO learning to accurately predict the next pitch the pitcher makes.

The top row of graphs in Fig. 5 ("Basic Features") shows the proportion of Clayton Kershaw's pitches (y-axis) correctly predicted by the AOs across each epoch (x-axis; one epoch means one pass through the pitcher's entire dataset). Both AOs were built to predict the next response using the above-described basic preparation, with one AO trained using no feedback for incorrect predictions (left column) and the second AO being trained using penalties for incorrect predictions (right column). Though the proportion of correct predictions made by the AOs did improve across epochs, the AOs correctly predicted only half of the next responses emitted by Clayton Kershaw, with no meaningful difference when not providing feedback or when using punishment for incorrect predictions.

Adding Molecular Information

Our behavior science domain knowledge helps us provide possible paths toward improvement. For instance, we can provide the AO with molecular information such as what behaviors led to reinforcement for the human over the last five pitches (second row, Fig. 5). Note that, in so doing, we have added computational complexity as the total number of game states the AO needed to distinguish increased to 96,768.³ Adding behavior science domain knowledge of molecular influences on behavior

³ Using the "stars and bars" theorem, the number of combinations is:

$$\binom{n+k-1}{k-1}$$

where n is the total number of draws (five pitches) and k is the number of categories we have (four different pitch types). Plugging in the values:

$$1,728 \times \binom{5+4-1}{4-1} = \binom{8}{3} = 1,728 \times \frac{8 \times 7 \times 6}{3 \times 2 \times 1} = 1,728 \times 56 = 96,768.$$

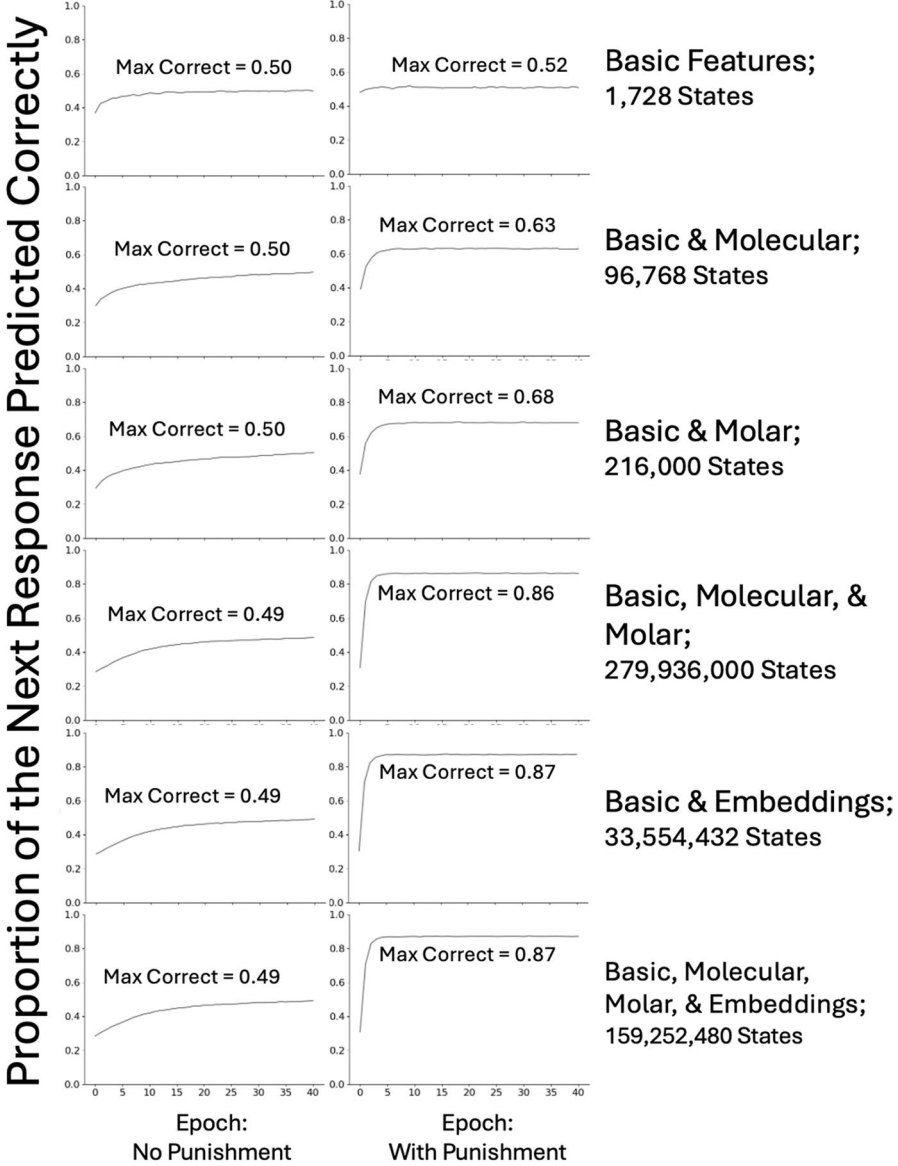


Fig. 5 Learning Curves for AOs Trained to Predict the Next Pitch Emitted by Clayton Kershaw Using the Game Variables and Reward Functions

increased the proportion of pitches correctly predicted by the AO contacting reinforcement and punishment to .63. It did not affect the proportion of pitches correctly predicted by the AO contacting reinforcement alone (.49).

Adding Molar Information

Another way we might add behavior science domain knowledge is by providing molar information to the AOs. To do this, beginning with pitch 500, we fit the generalized matching equation in a rolling manner over the previous 500 pitches (100 pitches per datum used to fit the model; third row, Fig. 5). We could then obtain the bias, sensitivity, and variance accounted for (VAC) by the generalized matching equation over the previous 500 pitches and use those as informational variables for the AO. Because these values are continuous, each was discretized into five bins to make the total game states computationally tractable (see supplemental code for more detail). In so doing, the number of game states the AO needed to distinguish increased to 216,000 ($1,728 \times 5^3$; five levels of the three categories from the GME). Adding behavior science domain knowledge on molar patterns of behavior increased the proportion of pitches correctly predicted by the AO contacting reinforcement and punishment to .68. Still, it did not affect the proportion of pitches correctly predicted by the AO contacting reinforcement alone (.50).

Adding Molecular and Molar Information

Despite molecular and molar accounts being theoretically at odds, both likely play a significant role in the behavior of biological organisms (e.g., Shimp, 2020). Thus, a third pair of AOs were trained using molecular and molar information, which led to 279,936,000 game states that the AO needed to distinguish. Combining molecular and molar information increased the proportion of the next responses correctly predicted by the AO to .86 in the reinforcement with punishment condition. As before, this did not affect the proportion of pitches correctly predicted by the AO contacting reinforcement alone (.49).

Embedding All Environmental Information

The pattern observed at this point was that AOs performed better as we provided more information about the environment in which the human was behaving. However, we had only used 7 out of the 61 available variables in the Statcast dataset. Including all 61 features, however, would have led to ~40.5 octillion (40,528,731,018,232,286,929,118,822,400) unique game states the AO would have had to distinguish. This highlights the curse of dimensionality common in RL research attempting to model nonlaboratory contexts. The curse of dimensionality often involves an exponential increase in computational complexity as the number of dimensions in a dataset increases (Sutton & Barto, 2018).

To work around this computational complexity, we created embeddings of all 61 features using an autoencoder (a type of neural network; e.g., Rama et al., 2021; see Appendix A for a full description and pseudocode). Embeddings were used to compress the 61 features into a lower-dimensional representation, reducing the number of game states to learn from 40.5 octillion to 33,554,432. When the embedded game states were combined with the molecular and molar information, the total number of game states increased to 159,252,480. The AO trained using only the embedded features performed identically to the AO trained using the embedded features with molecular and molar information. The proportion of next responses correctly predicted was .88 in the reinforcement with punishment condition (bottom two rows, Fig. 5). As with before, the proportion of next responses correctly predicted by AOs trained without punishment was still only .49.

Human Choice Modeled by Q-Learning

As a final demonstration of the utility of bridging the gap between behavior science and RL in AI, we can also ask whether we even need to build AOs to predict a human's next response. As exemplified by the matching law example in the second section above, the Q-learning algorithm is similar to behavioral economics in the foundational assumption that biological organisms associate reinforcement and punishment with emitting specific actions in specific contexts and that these learned associations build on each other over time to create the organism's learning history. Thus, rather than build an AO to predict responses and whose predictions we reinforce or punish, what might happen if we simply build a Q-table and let the human's experiences update the Q-table? To do this, we used the basic, molecular, and molar features, updated the Q-table with +1 if the pitcher contacted a strike or out and a -1 otherwise (Cox et al., 2017, 2021; Falligant et al., 2021). For Clayton Kershaw, the pitcher for whom the details of this section were reported, the proportion of next pitches correctly predicted using this approach was .92. That is, 92% of the time, this approach would correctly predict the next pitch thrown.

Replicating across Five More People

Tables 2 and 3 show the results of replicating the simulations described above for five additional professional MLB pitchers. In both tables, the rows correspond to the feature set used, whether punishment was involved, and the total number of environmental states the AO had to associate with action-reward pairs. Table 2 shows the accuracy of predictions made across the feature sets for each participant and the overall arithmetic mean (denoted AVG). Table 3 shows the Akaike Information Criterion (AIC) for the same models whose results are displayed in Table 2.

Using Q-learning in simple contexts with operant information for all six pitchers allowed us to predict the next pitch they threw with over 90% accuracy. Using reinforcement and punishment led to the highest accuracy regardless of the feature set

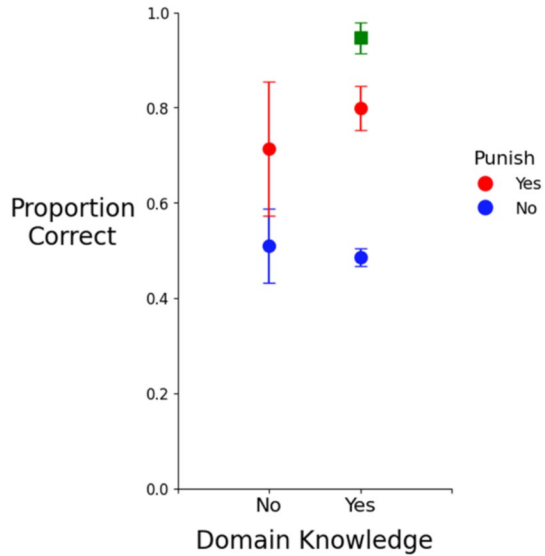
Table 2 Accuracy of Predictions across Feature Sets Sorted from Highest to Lowest Average Accuracy

Feature Set	Punish	Total States	AIC Rank	AVG	Kershaw	Bumgarner	Cole	Hamel	Linceum	Ohtani
Human as Q Table	--	143,327,232	7	0.95	0.92	0.9	0.96	0.94	0.97	0.99
Molecular + Molar + Embeddings	Yes	132,710,400	10	0.89	0.85	0.86	0.9	0.89	0.89	0.93
All Feats Embedded w/o Molecular & Molar	Yes	33,554,432	11	0.89	0.86	0.86	0.9	0.89	0.89	0.93
Molecular & Molar	Yes	279,936,000	8	0.89	0.86	0.85	0.89	0.88	0.9	0.93
Molar	Yes	216,000	3	0.73	0.68	0.74	0.69	0.68	0.66	0.9
Molecular	Yes	96,768	5	0.71	0.63	0.73	0.66	0.67	0.68	0.88
Simple Context	Yes	1,728	1	0.57	0.51	0.69	0.47	0.5	0.38	0.85
Most Frequent Pitch Thrown	--	N/A	--	0.57	0.6	0.78	0.64	0.56	0.35	0.46
Simple Context	No	1,728	2	0.54	0.5	0.67	0.46	0.48	0.37	0.78
Molecular	No	96,768	6	0.51	0.5	0.56	0.47	0.46	0.45	0.62
Molar	No	216,000	4	0.48	0.5	0.51	0.46	0.45	0.44	0.54
Molecular & Molar	No	279,936,000	9	0.48	0.49	0.44	0.48	0.44	0.5	0.5
Molecular + Molar + Embeddings	No	132,710,400	13	0.48	0.49	0.43	0.49	0.44	0.5	0.5
All Feats Embedded w/o Molecular & Molar	No	33,554,432	12	0.47	0.48	0.43	0.49	0.44	0.49	0.5
Random Choice Model	--	N/A	--	0.24	0.26	0.25	0.25	0.25	0.25	0.16

Table 3 AIC for the Models across All Feature Sets Sorted from the Average Lowest to Highest AIC

Feature Set	Punish	Total States	Accuracy Rank	AVG	Kershaw	Bungarner	Cole	Hamels	Linecum	Ohtani
Simple Context	Yes	1,728	7	8.769E+04	1.219E+05	9.868E+04	9.249E+04	1.104E+05	8.543E+04	1.716E+04
Simple Context	No	1,728	8	8.803E+04	1.261E+05	8.545E+04	9.424E+04	1.143E+05	8.637E+04	2.177E+04
Molar	Yes	216,000	5	9.098E+05	9.514E+05	9.331E+05	9.308E+05	9.433E+05	1.034E+06	6.658E+05
Molar	No	216,000	10	9.384E+05	9.861E+05	9.716E+05	9.595E+05	9.790E+05	1.057E+06	6.774E+05
Molecular	Yes	96,768	6	1.262E+07	1.252E+07	1.498E+07	1.497E+07	1.498E+07	1.679E+07	1.496E+06
Molecular	No	96,768	9	1.265E+07	1.254E+07	1.501E+07	1.500E+07	1.501E+07	1.682E+07	1.505E+06
Human as Q Table	--	143,327,232	1	8.047E+08	7.960E+08	9.552E+08	9.549E+08	9.546E+08	1.072E+09	9.550E+07
Molecular & Molar	Yes	279,936,000	4	8.047E+08	7.960E+08	9.552E+08	9.549E+08	9.546E+08	1.072E+09	9.550E+07
Molecular & Molar	No	279,936,000	11	8.048E+08	7.961E+08	9.552E+08	9.550E+08	9.547E+08	1.072E+09	9.551E+07
Molecular + Molar + Embeddings	Yes	132,710,400	2	9.538E+08	1.060E+09	1.273E+09	1.274E+09	1.273E+09	7.152E+08	1.272E+08
All Feats Embedded w/o Molecular & Molar	Yes	33,554,432	3	9.538E+08	1.060E+09	1.273E+09	1.274E+09	1.273E+09	7.152E+08	1.272E+08
All Feats Embedded w/o Molecular & Molar	No	33,554,432	13	9.538E+08	1.061E+09	1.273E+09	1.274E+09	1.273E+09	7.152E+08	1.272E+08
Molecular + Molar + Embeddings	No	132,710,400	12	9.538E+08	1.061E+09	1.273E+09	1.274E+09	1.273E+09	7.152E+08	1.272E+08

Fig. 6 Interaction of Adding Domain Knowledge and Punishment to the AOs' Ability to Correctly Predict the Next Response. *Note.* Error bar = 95% CI. The square is from modeling human choice directly via Q-learning



(also see Fig. 6). Adding behavior science domain expertise around choice improved performance beyond simple stimulus conditions (also see Fig. 6). Finally, we showed that one can get good predictive capabilities using a pure AI approach to include all contextual information in a computationally tractable way (via embeddings) without considering operant principles. However, we could reduce the assumptive complexity of the process by simply including knowledge about how human operant behavior works, which led to similar predictive performance. For example, the molar model with punishment led to the third lowest AIC with an average prediction accuracy of 0.73.

How Artificial Intelligence Can Benefit from Behavior Science

Despite the differences in how AI and behavior science researchers apply selectionist principles to shape and change behavior, both sides could likely gain from contacting the methods and findings from the other. One area where AI researchers would likely benefit from behavior scientists is the robust domain knowledge in the efficient and tailored use of reinforcement schedules to predict and control behavior. Behavior scientists are experts in identifying and using complicated reinforcement schedules in complicated, nonlaboratory contexts where AI systems are increasingly being deployed.

A second area where behavior science expertise may help is with the curse of dimensionality, referenced previously. We have many mathematical models that describe what organisms pay attention to, how that might change over time, and how an organism changes its allocation of behavior (Cox, 2023). As AI researchers build multimodal systems that perform more than one task and are embedded in dynamic

nonlaboratory environments, they likely will be unable to capture and attend to all stimuli in the surrounding environment as it would be too computationally expensive. As illustrated in the simulations, behavior science may help reduce the total feature space to make problems more computationally tractable by showing what biological organisms attend to and how it relates to the system's long-term survival.

Finally, behavior science domain knowledge may also be useful around complicated or complex goal setting. Biological systems have evolved to respond to many concurrent, mixed, multiple, and complicated schedules of reinforcement, punishment, and extinction relative to many important biological functions. AI systems are often focused on one or a few highly specific and well-defined tasks. As multimodal AI systems are built with increasingly more goals to optimize around, the literature on decision making and choice in biological organisms could prove useful. For example, it is unclear that biological organisms optimize reinforcement and minimize punishment in all contexts (e.g., Mazur, 1981). There is probably a reason this is so from an evolutionary perspective. Perhaps there is something here AI researchers should attend to when creating artificially intelligent systems that have the goal of a generalizable repertoire.

How Behavior Science Can Benefit from Artificial Intelligence

As behavior scientists, we can also gain a lot from RL researchers. First, RL researchers have provided simple, generalizable, and flexible algorithms optimized for current software and hardware, allowing us to run computational simulations about behavior-environment relations. It is easy to scale the complexity of the environment in these simulations (e.g., scaling from 1,728 to ~280 million in the present study). One can use whatever reinforcement or punishment contingencies one desires with no IRB or IACUC approval required. The relation between states (e.g., stimuli), consequences (e.g., schedules of reinforcement), and dynamically changing behavioral repertoires can be readily analyzed and scaled in complexity. And one can manipulate and formally test any theoretical or mechanistic approach desired to make precise, mathematical, and quantitative claims while constrained to all known behavioral principles and processes (e.g., molecular, molar, evolutionary theory of behavior dynamics, mathematical principles of reinforcement; McDowell, 2019; Killeen & Sitomer, 2003).

The above benefit should be considered a practical benefit, as well as a theoretical benefit. The demonstration above showed that this approach can describe and predict human behavior in nonlaboratory, dynamic settings. In practical terms, it is easy to envision a future where the environment and behavior data come from a clinical setting instead of a baseball game, and RL approaches are used to understand how a client or patient is learning and adapting their behavior over time. Once modeled, simulations could be run forward in time to understand how the client's behavior is likely to change if different contingencies of reinforcement exist,

different disruptions to existing schedules of reinforcement are introduced, and additional responses are added to the individual's repertoire. The output of such a RL approach would be identification of the ideal reinforcement schedules and additional responses needed within someone's repertoire such that responding persists in the face of disruption (when needed) or adapts appropriately (when needed). Though using a different approach, others have already begun exploring the clinical utility of computational modeling (e.g., Hagopian & Falligant, 2023).

A second benefit illustrated in the current simulation was how to integrate many areas of behavior science to model the behavior of whole organisms. We have no shortage of models highlighting what organisms pay attention to, how that might change over time, and what behavior–environment relations look like on many different levels (Cox, 2023). The algorithms and methods described in the RL literature allow researchers to combine these into a single integrated system, animating a whole organism constrained by known behavioral principles and processes. This opens the door to many novel research questions about how known behavioral processes, often studied in isolation, interact across time and space to influence the behavioral repertoires of whole organisms.

Lastly, computational simulations may offer a complementary method to describing and predicting behavior–environment relations and generating quantitative predictions around how to better control behavior. This approach has proven tremendously useful in many other scientific fields (e.g., computational physics), and ETBD is already adding to our understanding of behavior in behavior science (Hagopian & Falligant, 2023; McDowell, 2019). Also striking to us was the relative performance of models containing punishment versus those that did not. Interesting avenues for future work include better understanding how computational and simulation approaches might allow us to better understand punishment contingencies without having to expose biological organisms as well as how reinforcement schedule manipulations might lead to the same predictive performance without requiring punishment be present. Future research in RL in AI, as informed by behavior science, can further push the boundaries of our scientific knowledge into unexplored or underexplored territories.

Appendix 1

Dimensional Reduction via Autoencoders

An autoencoder is an unsupervised neural network architecture designed to learn how to represent high-dimensional data in lower dimensions (Hinton & Salakhutdinov, 2006). Autoencoders accomplish this through the two primary components of the autoencoder architecture: an *encoder* and a *decoder* (Fig. 7). The *encoder* compresses the input data into a lower-dimensional representation while preserving essential features of the input data. The *decoder* attempts to reconstruct the original input data at the output layer from the compressed representation created by the encoder

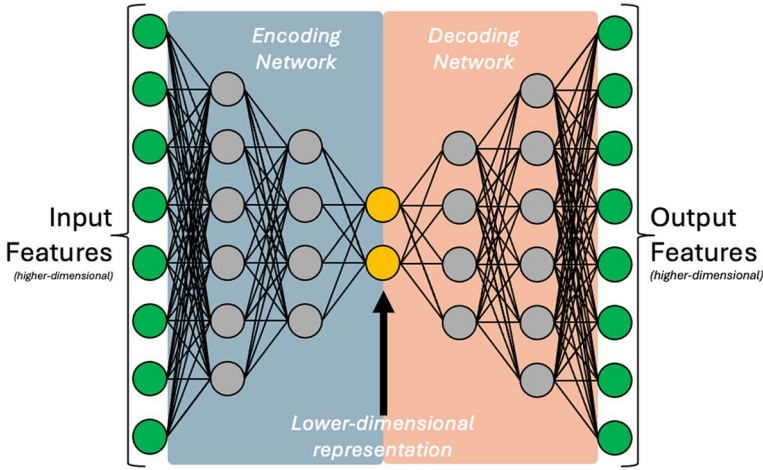


Fig. 7 Visual Representation of an Autoencoder Architecture

(Goodfellow et al., 2016). In other words, autoencoders force the input data through a lower-dimensional bottleneck. This requires the model to preserve the most important features of the data while discarding noise and redundant information.

Mathematical Formalism

We can formalize this mathematically. Given an input vector $\mathbf{x} \in \mathbb{R}^n$, the encoder function $f(\mathbf{x})$ maps the input vector to a lower-dimensional representation $\mathbf{z} \in \mathbb{R}^m$ (where $m < n$):

$$\mathbf{z} = f(\mathbf{x}) = \sigma(\mathbf{W}_e \mathbf{x} + \mathbf{b}_e).$$

Here, $f_\theta: \mathbf{X} \rightarrow \mathbf{Z}$ is parameterized by θ ; $\mathbf{W}_e \in \mathbb{R}^{m \times n}$ represents the encoding weight matrix; $\mathbf{b}_e \in \mathbb{R}^m$ represents the encoding bias vector; and $\sigma(\cdot)$ denotes the activation function (rectified linear units [ReLU] in the present simulation which is common for modern autoencoder implementations; Goodfellow et al., 2016). Next, given the output of the encoder function ($\mathbf{z} \in \mathbb{R}^m$), the decoder function attempts to recreate the higher-dimensional input vector ($\hat{\mathbf{x}} \in \mathbb{R}^n$):

$$\hat{\mathbf{x}} = g(\mathbf{z}) = \sigma'(\mathbf{W}_d \mathbf{z} + \mathbf{b}_d).$$

Here $g_\phi: \mathbf{Z} \rightarrow \hat{\mathbf{X}}$ is parameterized by ϕ ; $\mathbf{W}_d \in \mathbb{R}^{n \times m}$ represents the decoding weight matrix; $\mathbf{b}_d \in \mathbb{R}^n$ are the decoding parameters; and $\sigma'(\cdot)$ denotes the decoding activation function.

The autoencoder neural network parameters θ and ϕ are optimized to minimize the overall error between the input vector ($\mathbf{x}$) and the recreated output vector ($\hat{\mathbf{x}}$). This is typically expressed as the mean squared error (MSE) loss function (Vincent et al., 2010):

$$L(\mathbf{x}, \hat{\mathbf{x}}) = \|\hat{\mathbf{x}} - g_{\varphi}(f_{\theta}(\mathbf{x}))\|^2.$$

Optimization is often performed via stochastic gradient descent using the Adam (adaptive moment estimation) optimizer (Kingma & Ba, 2014), which adapts the learning rate for each individual parameter based on estimates of first and second moments of the gradients.⁴

Network Architecture

We used a symmetric deep autoencoder architecture with three encoding layers and three decoding layers. The basic concept of using autoencoders for dimensionality reduction was established by Hinton and Salakhutdinov (2006). However, our implementation incorporates several changes based on modern deep learning practices. In particular, we use ReLU activations (Nair & Hinton, 2010) instead of sigmoid units, and the Adam optimizer (Kingma & Ba, 2014) instead of standard stochastic gradient descent. We also trained the network end-to-end, which contrasts with earlier implementations that relied on layer-wise pretraining using restricted Boltzmann machines (RBMs). RBMs are probabilistic graphical models that were historically used to initialize network weights in a layer-by-layer fashion before final training (Hinton & Salakhutdinov, 2006). This pretraining strategy was crucial in the early days of deep learning to prevent vanishing gradients and ensure good optimization, but has largely been superseded by modern architectures, better optimization techniques, and improved activation functions (Goodfellow et al., 2016).

In particular, our autoencoder network architecture implemented the following structure. The encoder was comprised of: (1) an input layer of n dimensions from the original feature space; (2) the first hidden layer of 128 neurons using ReLU activation; (3) the second hidden layer of 64 neurons using ReLU activation; and (3) the bottleneck layer of two neurons using ReLU activation. The decoder was comprised of: (1) the first hidden layer of 64 neurons using ReLU activation; (2) the second hidden layer of 128 neurons using ReLU activation; and (3) the output layer of n dimensions to match the original feature space and using linear activation. This architecture follows best practices for deep autoencoders (Baldi, 2012; Bengio et al., 2013).

Pseudocode

The creation of embeddings follows a systematic process that incorporates both dimensionality reduction and discretization:

⁴ First moments of the gradient estimate the average gradient for each parameter, which helps define the direction the parameter should change within the parameter space. Second moments of the gradient estimate how much the gradient of each parameter is changing (uncentered variance), which helps determine the step size the parameter update should take.

```
``pseudocode
```

ALGORITHM 1: Embedding Generation

Input: High-dimensional feature matrix $\mathbf{X} \in \mathbb{R}^{n \times d}$

Output: Binary embedding matrix $\mathbf{E} \in \{0,1\}^{n \times k}$

```

1: // Data Preprocessing
2:  $\mathbf{X}_{\text{scaled}} \leftarrow \text{StandardScaler.fit\_transform}(\mathbf{X})$ 
3:
4: // Autoencoder Training
5:  $\text{autoencoder} \leftarrow \text{CreateAutoencoder}(\text{input\_dim}=d)$ 
6:  $\text{autoencoder.compile}(\text{optimizer}=\text{'adam'}, \text{loss}=\text{'mse'})$ 
7:  $\text{autoencoder.fit}(\mathbf{X}_{\text{scaled}}, \mathbf{X}_{\text{scaled}},$ 
8:      $\text{epochs}=100,$ 
9:      $\text{batch\_size}=1272,$ 
10:     $\text{validation\_split}=0.2)$ 
11:
12: // Embedding Extraction
13:  $\text{encoder} \leftarrow \text{ExtractEncoder}(\text{autoencoder})$ 
14:  $\mathbf{Z} \leftarrow \text{encoder.predict}(\mathbf{X}_{\text{scaled}})$ 
15:
16: // Discretization
17: for  $j$  in 1 to  $k$  do
18:   if  $\text{var}(\mathbf{Z}[:,j]) > 0$  then
19:      $\mathbf{E}[:,j] \leftarrow \text{Quantile\_Discretize}(\mathbf{Z}[:,j], \text{bins}=2)$ 
20:   end if
21: end for
22: return  $\mathbf{E}$ 
``
```

This pseudocode implementation incorporates several key considerations. First, we used z-score normalization to ensure that all input dimensions were consistently scaled (Jain & Dubes, 1988). Z-score normalization is accomplished by subtracting the arithmetic mean from each value and dividing by the standard deviation (i.e., $x_{\text{scaled}} = (x - \mu)/\sigma$). Our training configuration involved 100 epochs (i.e., one epoch is one complete pass through the dataset), a batch size of 1272 (i.e., the number of training samples processed before the model weights are updated), a validation size of 20% (i.e., the portion of the data withheld from training the model to use for evaluating the model performance), and the Adam optimizer with default parameters ($\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$). Last, the output embeddings were continuous and were, thus, discretized to provide a binary representation of the original feature space, capturing essential patterns while maintaining computational efficiency.

Author Contributions David J. Cox is responsible for the conceptual or design of the work; the acquisition, analysis, or interpretation of data; the creation of the code used in the work; and revised and edited the original through final versions of the manuscript. Carlos Santos is responsible for drafting and revising the original submission of the manuscript.

Funding None.

Data Availability The data used for the reinforcement learning simulations is available in a repository linked within the methods section.

Declarations

Conflicts of Interest None.

Clinical Trial Number Not applicable.

References

- Adler, J. (1966). Chemotaxis in bacteria. *Science*, 153(3737), 708–716. <https://doi.org/10.1126/science.153.3737.708>
- Albers, S. V., & Meyer, B. H. (2011). The archaeal cell envelope. *Nature Reviews Microbiology*, 9(6), 414–426. <https://doi.org/10.1038/nrmicro2576>
- Ayllon, T., & Michael, J. (1959). The psychiatric nurse as a behavioral engineer. *Journal of the Experimental Analysis of Behavior*, 2, 323–334. <https://doi.org/10.1901/jeab.1959.2-323>
- Baldi, P. (2012). Autoencoders, unsupervised learning, and deep architectures. In: *Proceedings of ICML workshop on unsupervised and transfer learning* (pp. 37–49). <https://proceedings.mlr.press/v27/baldi12a.html>
- Baluska, F., & Mancuso, S. (2009). Plant neurobiology: from sensory biology, via plant communication, to social plant behavior. *Cognitive Processing*, 10(Suppl 1), S3–S7. <https://doi.org/10.1007/s10339-008-0239-6>
- Banzhaf, W. (2009). Self-organizing systems. In R. Meyers (Ed.), *Encyclopedia of complexity and systems science* (pp. 8040–8050). Springer. <https://doi.org/10.1007/978-0-387-30440-3-475>
- Baum, W. M. (1974). On two types of deviation from the matching law: Bias and undermatching. *Journal of the Experimental Analysis of Behavior*, 22(1), 231–242.
- Bengio, Y., Courville, A., & Vincent, P. (2013). Representation learning: A review and new perspectives. *IEEE Transactions on Pattern Analysis & Machine Intelligence*, 35(8), 1798–1828. <https://doi.org/10.1109/TPAMI.2013.50>

- Cox, D. J. (May, 2023). Mapping the landscape of quantitative models of behavior-environment relations. In *45th Meeting of the Society for the Quantitative Analyses of Behavior*.
- Cox, D. J., Sosine, J., & Dallery, J. (2017). Application of the matching law to pitch selection in professional baseball. *Journal of Applied Behavior Analysis*, 50(2), 393–406. <https://doi.org/10.1002/jaba.381>
- Cox, D. J., Klapes, B., & Falligant, J. M. (2021). Scaling N from 1 to 1,000,000: Application of the generalized matching law to big data contexts. *Perspectives on Behavior Science*, 44, 641–665. <https://doi.org/10.1007/s40614-021-00298-8>
- Cox, D.J., Weil, L., Farrow, J., Jennings, A., Santos, C., & Sosine, J. (2023). Reinforcement learning in behavior science and artificial intelligence. In D. Crone-Todd, D. Hantula, & T. V. Layng (Eds.), *Behavior science and digital technologies*. Preprint Link: <https://psyarxiv.com/ue5vx>
- Davison, M., & Cowie, S. (2022). Modeling choice across time: Effects of response-reinforcer discriminability. *Journal of the Experimental Analysis of Behavior*, 117(1), 36–52. <https://doi.org/10.1002/jeab.723>
- Falligant, J. M., Cero, I., Kranak, M. P., & Kurtz, P. F. (2021). Further application of the generalized matching law to multialternative sports contexts. *Journal of Applied Behavior Analysis*, 54(1), 389–402.
- Farley, B. G., & Clark, W. A. (1954). Simulation of self-organizing systems by digital computer. *IRE Transactions on Information Theory*, 4, 76–84.
- Ferster, C. B., & Skinner, B. F. (1957). *Schedules of reinforcement*. Appleton-Century-Crofts.
- Finegan, J. (2019). *Archaeological history of ancient Middle East*. Taylor & Francis.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.
- Graesser, L., & Keng, W. L. (2019). *Foundations of deep reinforcement learning: Theory and practice in Python*. Addison-Wesley Professional.
- Hagopian, L. P., & Falligant, J. M. (2023). Application of the evolutionary theory of behavior dynamics to severe challenging behavior. *Journal of Applied Behavior Analysis*, 56(4), 729–744. <https://doi.org/10.1002/jaba.1018>
- Hawkins, R. D., Kandel, E. R., & Bailey, C. H. (2006). Molecular mechanisms of memory storage in Aplysia. *Biological Bulletin*, 210(3), 174–191. <https://doi.org/10.2307/4134556>
- Hedrich, R., & Neher, E. (2018). Venus flytrap: How an excitable, carnivorous plant works. *Trends in Plant Science*, 23(3), 220–234. <https://doi.org/10.1016/j.tplants.2017.12.004>
- Hinton, G. E., & Salakhutdinov, R. R. (2006). Reducing the dimensionality of data with neural networks. *Science*, 313(5786), 504–507. <https://doi.org/10.1126/science.1127647>
- Hursh, S. R., & Silberberg, A. (2008). Economic demand and essential value. *Psychological Review*, 115(1), 186–198. <https://doi.org/10.1037/0033-295X.115.1.186>
- Hutchison, W. R. (2012). The central role for behavior analysis in modern robotics, and vice versa. *The Behavior Analyst*, 35(1), 29–35. <https://doi.org/10.1007/BF03392262>
- Ifenthaler, D., & Seel, N. M. (2012). Stochastic models of learning. In N. M. Seel (Ed.), *Encyclopedia of the sciences of learning* (pp. 3192–3194). Springer. <https://doi.org/10.1007/978-1-4419-1428-6-502>
- Jain, A. K., & Dubes, R. C. (1988). *Algorithms for clustering data*. Prentice Hall.
- Kandel, E. R. (1979). *Behavioral biology of Aplysia: A contribution to the comparative study of opisthobranch mollusks*. Freeman.
- Killeen, P. R., & Sitomer, M. T. (2003). MPR. *Behavioural Processes*, 62, 49–64. [https://doi.org/10.1016/S0376-6357\(03\)00017-2](https://doi.org/10.1016/S0376-6357(03)00017-2)
- Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. *arXiv*. <https://doi.org/10.48550/arXiv.1412.6980>
- Kober, J., Bagnell, J. A., & Peters, J. (2013). Reinforcement learning in robotics: A survey. *International Journal of Robotics Research*, 32(11), 1238–1274. <https://doi.org/10.1177/0278364913495721>
- Major League Baseball. (2024). *savant*. https://baseballsavant.mlb.com/statcast_search
- Mazur, J. E. (1981). Optimization theory fails to predict performance of pigeons in a two-response situation. *Science*, 214(4522), 823–825. <https://www.jstor.org/stable/1686991>
- McDowell, J. J. (2019). On the current status of the evolutionary theory of behavior dynamics. *Journal of the Experimental Analysis of Behavior*, 111(1), 130–245. <https://doi.org/10.1002/jeab.495>
- Michie, D. (1961). *Trial and error Science Survey* (pp. 129–145). Penguin. <https://people.csail.mit.edu/brooks/idocs/matchbox.pdf>
- Minsky, M. L. (1954). *Theory of neural-analog reinforcement systems and its application to the Brain-Model Problem*. Ph.D. thesis. Princeton University. <https://www.proquest.com/docview/301998727>
- Minsky, M. L. (1961). Steps toward artificial intelligence. *Proceedings of the Institute of Radio Engineers*, 49, 8–30. <https://courses.csail.mit.edu/6.803/pdf/steps.pdf>

- Morgan, D. L. (2010). Schedules of reinforcement at 50: A retrospective appreciation. *Psychological Record*, 60, 151–172. <https://doi.org/10.1007/BF03395699>
- Public Broadcasting Systems. (2024). Baseball for beginners. <https://www.pbs.org/kenburns/baseball/baseball-for-beginners>
- Rachlin, H. (2006). Notes on discounting. *Journal of the Experimental Analysis of Behavior*, 85(3), 425–435. <https://doi.org/10.1901/jeab.2006.85-05>
- Rama, K., Kumar, P., & Bhasker, B. (2021). Deep autoencoders for feature learning with embeddings for recommendations: A novel recommender system solution. *Neural Computing & Applications*, 33, 14167–14177. <https://doi.org/10.1007/s00521-021-06065-9>
- Sammut, C. (2011). Credit assignment. In C. Sammut & G. I. Webb (Eds.), *Encyclopedia of machine learning* (pp. 238–242). Springer. <https://doi.org/10.1007/978-0-387-30164-8-185>
- Samuel, A. L. (1959). Some studies in machine learning using the game of checkers. *IBM Journal of Research & Development*, 3(3), 211–229. <https://doi.org/10.1147/rd.33.0210>
- Shimp, C. P. (2020). Molecular (moment-to-moment) and molar (aggregate) analyses of behavior. *Journal of the Experimental Analysis of Behavior*, 114(3), 394–429. <https://doi.org/10.1002/jeab.626>
- Silver, D., Huang, A., Maddison, C. J., ..., Hassabis, D. (2016). Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587), 484–489. <https://doi.org/10.1038/nature16961>
- Skinner, B. F. (1938). *The behavior of organisms: An experimental analysis*. Appleton-Century.
- Staddon, J. E. R. (2001). *Adaptive dynamics: The theoretical analysis of behavior*. Bradford Books.
- Staddon, J. E. R., & Zhang, Y. (2017). On the assignment-of-credit problem in operant learning. In M. L. Commons, S. Grossberg, & J. Staddon (Eds.), *Neural network models of conditioning and action* (pp. 279–293). Routledge.
- Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2nd ed.). Bradford Books.
- Tesauro, G. (1995). TD-Gammon: A self-teaching backgammon program. In A. F. Murray (Ed.), *Applications of neural networks* (pp. 267–285). Springer. <https://doi.org/10.1007/978-1-4757-2379-3-11>
- Thorndike, E. L. (1898). Animal intelligence: An experimental study of the associative processes in animals. *Psychological Review, Monograph Supplements*, 2, 4–160. <https://doi.org/10.1037/10780-000>
- Touretzky, D. S., & Saksida, L. M. (1997). Operant conditioning in Skinnerbots. *Adaptive Behavior*, 5(3–4), 219–247. <https://doi.org/10.1177/105971239700500302>
- Valentine, D. L. (2007). Adaptations to energy stress dictate the ecology and evolution of the Archaea. *Nature Reviews Microbiology*, 5(4), 316–323. <https://doi.org/10.1038/nrmicro1619>
- Vincent, P., Larochelle, H., Lajoie, I., Bengio, Y., & Manzagol, P. A. (2010). Stacked denoising autoencoders: Learning useful representations in a deep network with a local denoising criterion. *Journal of Machine Learning Research*, 11, 3371–3408. <https://www.jmlr.org/papers/v11/vincent10a.html>
- Watkins, C. J. C. H., & Dayan, P. (1992). Q-learning. *Machine Learning*, 8, 279–292. <https://doi.org/10.1007/BF00992698>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Authors and Affiliations

David J. Cox^{1,2}  · Carlos Santos¹

✉ David J. Cox
dcox@endicott.edu

¹ Institute of Applied Behavioral Science at Endicott College, Beverly, MA, USA

² Mosaic Pediatric Therapy, Charlotte, NC, USA